

STACK QNO:5

QUESTION:

Problem Description:

Sajid was booking a tour package of IRCTC from Chennai to Delhi for his family.

Two of the relatives was interested in joining to this tour.

these two persons are studying engineering in computing technology, only one tickets are remaining in the IRCTC portal.

So, Sajid decided to book one ticket for out of those persons also along with his family members.

He wants to identify the one person out of these persons. he decided to conduct a technical task to identify the right person to travel.

the task was that, Check for Balanced Brackets in an expression using Stack

Can you help them to complete the task?

Function Description

- Declare a character stack S.
- Now traverse the expression string exp.
 1. If the current character is a starting bracket ('{' or '[' or '{') then push it to stack.
 2. If the current character is a closing bracket (')' or ']' or '}') then pop from stack and if the popped character is the matching starting bracket then fine else brackets are not balanced.
- After complete traversal, if there is some starting bracket left in stack then "not balanced"

Input Format

Single line represents only braces (both curly and square)

Output Format

If the given input balanced then print as Balanced [or] Not Balanced

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN)	INPUT (STDIN)
{()}[]	{{{}}}[]
EXPECTED OUTPUT	EXPECTED OUTPUT
Balanced	Not Balanced

Mandatory Test Cases

Test Case 1	Test Case 2	Test Case 3
KEYWORD	KEYWORD	KEYWORD
pop()	empty()	push

Test Case 4
KEYWORD
stack

Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
15	275	100

Source Code:

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
#define MAX 1000
char stack[MAX];
int top = -1;
void push(char c) {
    if (top < MAX - 1) {
        top++;
        stack[top] = c;
    }
}

char pop() {
    if (top == -1) {
        return '\0';
    }
    char c = stack[top];
    top--;
    return c;
}

bool empty() {
    return top == -1;
}
```

```

bool isMatching(char char1, char char2) {
    if (char1 == '(' && char2 == ')') return true;
    if (char1 == '{' && char2 == '}') return true;
    if (char1 == '[' && char2 == ']') return true;
    return false;
}

int main() {
    char exp[1005];
    if (scanf("%s", exp) != 1) {
        return 0;
    }
    bool isBalanced = true;
    for (int i = 0; i < strlen(exp); i++) {
        if (exp[i] == '(' || exp[i] == '{' || exp[i] == '[') {
            push(exp[i]);
        }
        else if (exp[i] == ')' || exp[i] == '}' || exp[i] == ']') {
            if (empty()) {
                isBalanced = false;
                break;
            }
            char popped = pop();
            if (!isMatching(popped, exp[i])) {
                isBalanced = false;
                break;
            }
        }
    }

    if (isBalanced && empty()) {
        printf("Balanced\n");
    } else {
        printf("Not Balanced\n");
    }
    return 0;
}

```

Output:

```
{ () } []
Balanced
```

```
{ ({} ) ] }
Not Balanced
```